

EventLink

Assignment 3

Team members

Nuriel Taha
Amine Yesfi
Yilikal Sarka
Bekzat Arystanov

Introduction

Below you will find twelve **SELECT** queries for our EventLink platform. The set satisfies the assignment requirements: it contains multiple joins overall, several aggregation functions, and multiple GROUP BY/HAVING clauses. Each query is central to functionality we will implement. The SQL scripts are also included in `queries.sql` within the same zip. The database `db_ntaha` is already seeded with the necessary data.

1.1 Upcoming events in 2025 with at least 5 available seats

SQL

```
SELECT  e.event_id, e.title, v.name AS venue_name, e.start_at,
        COUNT(t.ticket_id) AS total_tickets,
        SUM(CASE WHEN t.status = 'AVAILABLE' THEN 1 ELSE 0 END) AS
        available_tickets
FROM Event e
JOIN Venue v  ON v.venue_id = e.venue_id
JOIN Ticket t ON t.event_id = e.event_id
WHERE e.start_at BETWEEN '2025-01-01 00:00:00' AND '2025-12-31 23:59:59'
        AND e.title LIKE '%'
GROUP BY e.event_id, e.title, v.name, e.start_at
HAVING available_tickets >= 5
ORDER BY e.start_at;
```

Output

event_id	title	venue_name	start_at	total_tickets	available_tickets
4	Indie Fest	Open Field Park	2025-09-15 16:00:00	15	9
5	Comedy Night	City Theatre	2025-10-10 20:30:00	15	15
1	Rock Night	Grand Arena	2025-10-20 20:00:00	15	5
2	Shakespeare in the Park	City Theatre	2025-11-05 19:00:00	15	9
3	Championship Finals	Grand Arena	2025-12-01 18:00:00	15	12

1.2 Seating map for event_id = 1 with ticket status and buyer if sold

SQL

```
SELECT  s.seat_id, s.section, s.row_label, s.seat_number, s.seat_label,
        COALESCE(t.status, 'NO_TICKET_DEFINED') AS ticket_status,
        t.ticket_id, t.price, t.qr_code,
        o.order_id, u.user_id, u.email, u.name
FROM Event e
JOIN Venue v  ON v.venue_id = e.venue_id
JOIN Seat s    ON s.venue_id = v.venue_id
LEFT JOIN Ticket t ON t.event_id = e.event_id AND t.seat_id = s.seat_id
LEFT JOIN 'Order' o ON o.order_id = t.order_id
LEFT JOIN 'User' u ON u.user_id = o.user_id
WHERE e.event_id = 1
ORDER BY s.section, s.row_label, s.seat_number;
```

Output

seat_id	section	row_label	seat_number	seat_label	ticket_status	ticket_id	price	qr_code	order_id	user_id	email	name
1	A	A	1	A1	TICKETED	1	50	QR1.1	1	1	alice@example.com	Alice
2	A	A	2	A2	TICKETED	8	50	QR1.2	1	1	alice@example.com	Alice
3	A	A	3	A3	TICKETED	9	50	QR1.3	1	1	alice@example.com	Alice
4	A	A	4	A4	TICKETED	10	50	QR1.4	1	1	alice@example.com	Alice
5	A	A	5	A5	TICKETED	11	50	QR1.5	1	1	alice@example.com	Alice
6	A	A	6	A6	TICKETED	12	50	QR1.6	1	1	alice@example.com	Alice
7	A	A	7	A7	TICKETED	13	50	QR1.7	1	1	alice@example.com	Alice
8	A	A	8	A8	TICKETED	14	50	QR1.8	1	1	alice@example.com	Alice
9	A	A	9	A9	AVAILABLE	15	50	QR1.9	NULL	NULL	NULL	NULL
10	A	A	10	A10	AVAILABLE	2	50	QR1.10	NULL	NULL	NULL	NULL
11	A	A	11	A11	AVAILABLE	3	50	QR1.11	NULL	NULL	NULL	NULL
12	A	A	12	A12	AVAILABLE	4	50	QR1.12	NULL	NULL	NULL	NULL
13	A	A	13	A13	AVAILABLE	5	50	QR1.13	NULL	NULL	NULL	NULL
14	A	A	14	A14	HELD	6	50	QR1.14	NULL	NULL	NULL	NULL
15	A	A	15	A15	HELD	7	50	QR1.15	NULL	NULL	NULL	NULL

1.3 Events with sell-through below 80%

SQL

```

SELECT e.event_id, e.title, e.start_at,
       COUNT(*) AS total_tickets,
       SUM(CASE WHEN t.status = 'TICKETED' THEN 1 ELSE 0 END) AS sold_tickets
FROM Event e
JOIN Ticket t ON t.event_id = e.event_id
GROUP BY e.event_id, e.title, e.start_at
HAVING sellthrough_ratio < 0.8
ORDER BY e.start_at;

```

Output

event_id	title	start_at	total_tickets	sold_tickets	sellthrough_ratio
4	Indie Fest	2025-09-15 16:00:00	15	6	0.4000
5	Comedy Night	2025-10-10 20:30:00	15	0	0.0000
1	Rock Night	2025-10-20 20:00:00	15	8	0.5333
2	Shakespeare in the Park	2025-11-05 19:00:00	15	4	0.2667
3	Championship Finals	2025-12-01 18:00:00	15	0	0.0000

2.1 Orders for user_id = 1 with ticket counts and venues

SQL

```

SELECT o.order_id, o.created_at, o.status, o.payment_status,
       COUNT(t.ticket_id) AS ticket_count,
       GROUP_CONCAT(DISTINCT v.name ORDER BY v.name SEPARATOR ', ') AS venues
FROM `Order` o
JOIN Ticket t ON t.order_id = o.order_id
JOIN Event e ON e.event_id = t.event_id

```

```

JOIN Venue v ON v.venue_id = e.venue_id
WHERE o.user_id = 1
GROUP BY o.order_id, o.created_at, o.status, o.payment_status
ORDER BY o.created_at DESC;

```

Output

order_id	created_at	status	payment_status	ticket_count	venues
1	2025-10-08 15:33:04	PAID	SUCCESS	8	Grand Arena

2.2 Order detail for order_id = 1 (tickets with seat and event info)

SQL

```

SELECT t.ticket_id, t.price, s.seat_label, e.title, e.start_at, v.name AS
venue_name
FROM Ticket t
JOIN `Order` o ON o.order_id = t.order_id
JOIN Event e ON e.event_id = t.event_id
JOIN Seat s ON s.seat_id = t.seat_id
JOIN Venue v ON v.venue_id = e.venue_id
WHERE o.order_id = 1
ORDER BY e.start_at, s.seat_label;

```

Output

ticket_id	price	seat_label	title	start_at	venue_name
1	50	A1	Rock Night	2025-10-20 20:00:00	Grand Arena
8	50	A2	Rock Night	2025-10-20 20:00:00	Grand Arena
9	50	A3	Rock Night	2025-10-20 20:00:00	Grand Arena
10	50	A4	Rock Night	2025-10-20 20:00:00	Grand Arena
11	50	A5	Rock Night	2025-10-20 20:00:00	Grand Arena
12	50	A6	Rock Night	2025-10-20 20:00:00	Grand Arena
13	50	A7	Rock Night	2025-10-20 20:00:00	Grand Arena
14	50	A8	Rock Night	2025-10-20 20:00:00	Grand Arena

2.3 Revenue per event (paid and successful orders only)

SQL

```

SELECT e.event_id, e.title, e.start_at,
SUM(t.price) AS gross_revenue,
COUNT(*) AS tickets_count
FROM Event e
JOIN Ticket t ON t.event_id = e.event_id
JOIN `Order` o ON o.order_id = t.order_id
WHERE t.status = 'TICKETED'
AND o.status = 'PAID'
AND o.payment_status = 'SUCCESS'

```

```
GROUP BY e.event_id, e.title, e.start_at
ORDER BY e.start_at;
```

Output

event_id	title	start_at	gross_revenue	tickets_count
4	Indie Fest	2025-09-15 16:00:00	210	6
1	Rock Night	2025-10-20 20:00:00	400	8
2	Shakespeare in the Park	2025-11-05 19:00:00	160	4

3.1 Sold-out events with waitlist size

SQL

```
SELECT e.event_id, e.title, e.start_at,
       COUNT(t.ticket_id) AS total_tickets,
       SUM(CASE WHEN t.status = 'AVAILABLE' THEN 1 ELSE 0 END) AS
       available_tickets,
       COALESCE(w.wl_size, 0) AS waitlist_size
FROM Event e
JOIN Ticket t ON t.event_id = e.event_id
LEFT JOIN (
  SELECT event_id, COUNT(*) AS wl_size
  FROM Waitlist
  WHERE status IN ('ACTIVE', 'OFFERED')
  GROUP BY event_id
) w ON w.event_id = e.event_id
GROUP BY e.event_id, e.title, e.start_at, w.wl_size
HAVING available_tickets = 0
ORDER BY e.start_at;
```

Output

No event is fully sold out in the current dataset, so the result is empty.

3.2 Next in line on the waitlist for event_id = 1

SQL

```
SELECT wl.entry_id, wl.created_at, wl.seats_requested,
       u.user_id, u.email, u.name
FROM Waitlist wl
JOIN 'User' u ON u.user_id = wl.user_id
WHERE wl.event_id = 1
      AND wl.status = 'ACTIVE'
ORDER BY wl.created_at ASC
LIMIT 1;
```

Output

entry_id	created_at	seats_requested	user_id	email	name
1	2025-10-08 15:33:04	2	1	alice@example.com	Alice

3.3 Held seats for event.id = 1 expiring within 30 minutes

SQL

```
SELECT t.ticket_id, s.seat_label, t.hold_expires_at
FROM Ticket t
JOIN Seat s ON s.seat_id = t.seat_id
WHERE t.event_id = 1
      AND t.status = 'HELD'
      AND t.hold_expires_at BETWEEN NOW() AND DATE_ADD(NOW(), INTERVAL 30 MINUTE)
ORDER BY t.hold_expires_at ASC;
```

Output

At the time of execution there were no held seats on event 1 expiring within the next 30 minutes, so the result is empty.

4.1 Category summary in 2025 (events, tickets created, tickets sold)

SQL

```
SELECT e.category,
       COUNT(DISTINCT e.event_id) AS events_count,
       COUNT(t.ticket_id)         AS tickets_created,
       SUM(CASE WHEN t.status = 'TICKETED' THEN 1 ELSE 0 END) AS tickets_sold
FROM Event e
LEFT JOIN Ticket t ON t.event_id = e.event_id
WHERE e.start_at BETWEEN '2025-01-01 00:00:00' AND '2025-12-31 23:59:59'
GROUP BY e.category
HAVING events_count > 0
ORDER BY tickets_sold DESC, e.category;
```

Output

category	events_count	tickets_created	tickets_sold
Concert	2	30	14
Theatre	2	30	4
Sports	1	15	0

4.2 Venue utilization in 2025 (tickets created vs. sold)

SQL

```
SELECT v.venue_id, v.name AS venue_name,
       COUNT(t.ticket_id) AS tickets_created,
       SUM(CASE WHEN t.status = 'TICKETED' THEN 1 ELSE 0 END) AS tickets_sold
FROM Venue v
JOIN Event e ON e.venue_id = v.venue_id
JOIN Ticket t ON t.event_id = e.event_id
WHERE e.start_at BETWEEN '2025-01-01 00:00:00' AND '2025-12-31 23:59:59'
GROUP BY v.venue_id, v.name
HAVING tickets_created > 0
ORDER BY tickets_sold DESC, v.name;
```

Output

venue_id	venue_name	tickets_created	tickets_sold
1	Grand Arena	30	8
3	Open Field Park	15	6
2	City Theatre	30	4

4.3 Integrity check: ticketed but unpaid

SQL

```
SELECT  t.ticket_id, e.title, s.seat_label,
        o.order_id, o.status AS order_status, o.payment_status
FROM    Ticket t
JOIN    Event e    ON e.event_id = t.event_id
JOIN    Seat s      ON s.seat_id = t.seat_id
LEFT JOIN `Order` o ON o.order_id = t.order_id
WHERE   t.status = 'TICKETED'
        AND (o.order_id IS NULL OR o.status <> 'PAID' OR o.payment_status <> '
        SUCCESS')
ORDER BY e.start_at, s.seat_label;
```

Output

All ticketed tickets in this dataset belong to paid and successful orders, so there are no inconsistencies to report.